

AY 2025/2026



POLITECNICO
MILANO 1863

Internet Of Things: Challenge 2

Part 1

Alberto Biffi - 10677341

Professor
Matteo CESANA Antonio BOIANO Fabio PALMESE

Version 1.0
April 23, 2026

Contents

1	Challenge Questions 1-2	2
1.1	CQ1a	2
1.2	CQ1b	3
1.3	CQ2	3
2	Challenge Questions (3-4)	6
2.1	CQ3a	6
2.2	CQ3b	6
2.3	CQ4	6
3	Challenge Questions (5-7)	7
3.1	CQ5	7
3.2	CQ6a	8
3.3	CQ6b	8
3.4	CQ7	9
4	Challenge Questions 8	9
4.1	CQ8	9

1 Challenge Questions 1-2

1.1 CQ1a

Find all NON Confirmable DELETE requests directed to the coap.me server that received a successful response. Take note of the MID of such requests.

First, we have to find the 'coap.me' server IP. We can do that by sending a ping to the server via the Windows CMD. We found out that its IPv4 is 134.102.218.18.

Now we apply this filter in Wireshark.

```
1 coap.code == 4 && coap.type == 1 && ip.dst == 134.102.218.18
```

The 'coap.code' filter only the DELETE method than we use 'coap.type' to filter only the NON Confirmable, and finally we filter with the 'ip.dst' with that of the coap.me server.

This results in 20 messages, with this MID and Token

MID	Token
1059	b651a74c2a74aecd
11787	21a587c5a0ae8a6e
21028	75912d3b51bc5016
22836	7f09e3677ebbc42
24716	e0864998dd738206
25814	8b6f78cbe076f916
30212	252ef436b7efa3ea
30800	880f30d8b543d6f7
32242	7801af5ea0b348f0
32298	7359ba9493b39e78
33264	7b9376a6a9484b8b
38052	2301e4de4156749f
38480	57f987c179a32d78
47220	1e6083b696fb3231
49932	452a18979a587044
53878	030bda63ff31592a
57936	e93dbbcb6a598a7d
62535	737051f79ba1e4d2
63041	f35bbf03a5ff38cc
63788	07f20b6ad047d749

Then we use those tokens to find out the responses of the coap.me server, and we can see that only the one with MID 30800 and token 880f30d8b543d6f7 has a successful response, all the other had a 'Method Not Allowed' response.

1.2 CQ1b

How many of the requests found in point CQ1a) actually obtained the desired outcome?

The request that we found had not obtained the desired outcome because after the delete another get (its token is 38f248207ef50a87) at the same Uri-Path have a 2.05 response, so the resource is still available.

1.3 CQ2

How many CoAP resources in the local server received the same number of unique unsuccessful Confirmable POST and PUT requests?

Assuming a resource receives X different CON unsuccessful POST requests and Y different CON unsuccessful PUT requests, how many resources have X=Y, with X>0?

In the following page, we have all of our Confirmable POST and PUT requests with the resource and their MID.

Found with this filter.

```
1 (coap.code == 2 || coap.code == 3) && coap.type == 0 && ip.dst == 127.0.0.1
```

MID	Uri-Path	CODE
50477	basic	PUT
42780	basic	POST
11242	basic	POST
40958	dining_room	POST
10680	dining_room	PUT
33760	dining_room/door	PUT
9853	dining_room/door	POST
10179	dining_room/door	POST
10181	dining_room/door	PUT
37991	dining_room/door	PUT
48174	dining_room/temperature	PUT
21336	dining_room/temperature	POST
53552	dining_room/temperature	POST
11409	dining_room/temperature	POST
10465	dining_room/temperature	POST
10466	dining_room/temperature	POST
40207	dining_room/temperature	PUT
58468	hello_post	POST
40964	hello_post	POST
40965	hello_post	POST
40939	hello_post	PUT
2579	hello_world	POST
26063	hello_world	POST
29078	hello_world	POST
57914	hello_world	POST
40534	living_room	POST
6326	living_room/door	PUT
21082	living_room/temperature	POST
30514	main_door	PUT
33536	main_door	PUT
1330	main_door	POST
64080	test	POST
6295	test	POST

We can exclude the `hello_world` and the test resources because all of the requests are POST, so we can not have $X = Y$.

Also, we can exclude `living_room`, `living_room/door`, and `living_room/temperature` because they only receive one request each, so we can not have $X = Y$ with $X > 0$.

Now we can look for the local server response, looking for the same MID, and find out how many POST and PUT are unsuccessful.

```
1 coap.mid == MID
```

This is the result after analyzing all packets.

Uri-Path	Unsuccessful POST	Unsuccessful PUT
<code>basic</code>	2	1
<code>dining_room</code>	1	1
<code>dining_room/door</code>	2	1
<code>dining_room/temperature</code>	5	2
<code>hello_post</code>	0	0
<code>main_door</code>	1	2

As we can see from the table, the only one with $X = Y$ and $X > 0$ is the `dining_room` resource.

2 Challenge Questions (3-4)

2.1 CQ3a

How many separate observe notifications have been sent containing values for the resource `"/dining_room/temperature"`, ignoring retransmissions?

First, we filtered the traffic to find all GET requests directed to the Uri-Path `"dining_room/temperature"`.

```
1 coap.opt.uri_path == "dining_room" && coap.opt.uri_path == "temperature" &&  
  coap.code == 1
```

We found 4 of such requests, but only one with the Observe option set to 0. Its MID is 10588, and its Token is 89b9.

We can use this Token to find how many separate observe notifications have been sent containing values.

```
1 coap.token == 89b9 && coap.opt.observe
```

We found a total of 25 messages. One is the request itself, and one is the piggybacked response received after the observe request. 9 of those are ICMP destination unreachable messages. And 4 of those are retransmissions.

The remaining 10 are the separate observe notifications we are looking for.

2.2 CQ3b

How many of the notifications found in point CQ3a) are useless (e.g., a waste of traffic)?

Of those 10 notifications, we can consider 5 of them useless because they arrived while the client was not connected.

2.3 CQ4

How many MQTT-SN messages are received by the clients from the local broker listening on port 1885?

After configuring the port to analyze MQTT-SN packets to 1885 we search for the packets sent from the local broker to the clients.

```
1 mqttsn && udp.srcport == 1885
```

This results in 0 packets displayed.

3 Challenge Questions (5-7)

3.1 CQ5

How many MQTT subscribers receive a last will message derived from a subscription that includes atleast one wildcard?

Firstly, we look for the connection with the last will active.

```
1 mqtt.conflag.willflag == 1
```

We have 4 will.

Will Topic	Will Message
university/department12/room1/temperature	error: a VIP Client just died
metaverse/room2/floor4	error: peuidql
hospital/facility3/area3	error: xlzaqtd
metaverse/room2/room2	error: zjzwrmdp

Now we search for all the publish sent with the uri being one of the 4 resources containing a will, and the will message.

```
1 mqtt.msgtype == 3 && tcp.srcport == 1883 && mqtt.topic == "university/
  department12/room1/temperature" && mqtt.msg contains "error: a VIP Client
  just died"
2 mqtt.msgtype == 3 && tcp.srcport == 1883 && mqtt.topic == "metaverse/room2/
  floor4" && mqtt.msg contains "error: peuidql"
3 mqtt.msgtype == 3 && tcp.srcport == 1883 && mqtt.topic == "hospital/facility3/
  area3" && mqtt.msg contains "error: xlzaqtd"
4 mqtt.msgtype == 3 && tcp.srcport == 1883 && mqtt.topic == "metaverse/room2/
  room2" && mqtt.msg contains "error: zjzwrmdp"
```

We found out that the only will that has been sent is the one with the 'university' topic.

And has been received by these 4 clients with ports 39551, 53557, 51743, and 41789.

We can now look for the Subscribe Request sent by these 4 clients containing the university topic.

```
1 mqtt.msgtype == 8 && (tcp.srcport == 39551 || tcp.srcport == 53557 || tcp.
  srcport == 41789 || tcp.srcport == 51743) && mqtt.topic contains "
  university"
```

We can see that the only client with a wildcard is the one on Port 51743 and with Client ID zmjnxudohrkaegmh.

3.2 CQ6a

How many different MQTT clients sent at least one message to erase a previously retained value in a topic in the public HiveMQ broker?

We used this query to find the HiveMQ broker IPs.

```
1 dns && dns.qry.name contains "hivemq"
```

We have 3 IPs to look for: 18.192.151.104, 35.158.34.213, and 35.158.43.69.

We can use this filter to find all the Publish packets without a payload. No payload means they are erasing a previously retained value.

```
1 mqtt.msgtype == 3 && mqtt.retain == 1 && tcp.dstport == 1883 && (ip.dst ==  
    18.192.151.104 || ip.dst == 35.158.34.213 || ip.dst == 35.158.43.69) && len  
    (mqtt.msg) == 0
```

We find 5 different clients with this Port and Client ID.

Port	Client ID
34779	pohujuv
36777	pcbtjyj
38207	giuxfijwus
47753	rolsmcwhshlbc
49877	atqeb1w

3.3 CQ6b

How many of the clients found in CQ6a) connected to the HiveMQ broker with a client identifier strictly longer than 7 bytes (>7)?

There are only two clients with a Client ID longer than 7 bytes. They are the giuxfijwus and the rolsmcwhshlbc clients.

3.4 CQ7

How many MQTT subscribe requests directed to the local broker specify a topic with at least two wildcards?

We can find them with just one filter, we have to filter only the subscribe requests sent to the local broker and then add a REGEX to that filter to find all of the topics with at least 2 wildcards.

```
1 mqtt.msgtype == 8 && (ip.dst == 127.0.0.1 || ipv6.dst == ::1) && mqtt.topic
  matches "[+#].*[+#]"
```

This gives us 9 different requests.

4 Challenge Questions 8

4.1 CQ8

Compare the local broker usage in the two capture files A.pcapng and B.pcapng. The goal is to have a plot on the number of layers of the topics used in the publish messages directed to the local broker in the two captures.

Produce a histogram plot with number of messages for each number of layers.

First we use a filter that gives us all the publish messages directed to the local broker

```
1 mqtt.msgtype == 3 && (ip.dst == 127.0.0.1 || ipv6.dst == ::1) && tcp.dstport ==
  1883
```

Now we use tshark to create a .txt file containing a list of all topics, one for each of our files A and B.

```
1 tshark -r A.pcapng -Y "mqtt.msgtype == 3 && (ip.dst == 127.0.0.1 || ipv6.dst ==
  ::1) && tcp.dstport == 1883" -T fields -e mqtt.topic > topics_A.txt
2 tshark -r B.pcapng -Y "mqtt.msgtype == 3 && (ip.dst == 127.0.0.1 || ipv6.dst ==
  ::1) && tcp.dstport == 1883" -T fields -e mqtt.topic > topics_B.txt
```

The last thing to do is to count the amount of layers in each file. We used Python to do that.

```
1 def conta_slash_nel_file(percorso_file):
2     # Numero di slash, quante righe lo hanno
3     distribuzione_slash = {}
4
5     try:
6         with open(percorso_file, 'r') as file:
7             for riga in file:
8                 riga = riga.strip()
9                 if riga:
10                     numero_slash = riga.count('/')
11                     if numero_slash in distribuzione_slash:
12                         distribuzione_slash[numero_slash] += 1
13                     else:
14                         distribuzione_slash[numero_slash] = 1
15
16     print(f"--- {percorso_file} ---")
```

```

17     for slash in sorted(distribuzione_slash.keys()):
18         # Se ho 1 slash -> 2 livelli, 2 slash -> 3 livelli etc.
19         livelli = slash + 1
20         print(f"N. Livelli: {livelli}: {distribuzione_slash[slash]}
21         messaggi")
22     except FileNotFoundError:
23         print(f"Errore")
24
25 # Esecuzione
26
27 nome_del_tuo_file = 'topics_A.txt'
28 conta_slash_nel_file(nome_del_tuo_file)
29
30 nome_secondo_file = 'topics_B.txt'
31 conta_slash_nel_file(nome_secondo_file)

```

These are the results. The total amount of packets analyzed is 439 for file A and 534 for file B.

File A

# of topic levels	# of PUBLISH
1	0
2	154
3	131
4	154

File B

# of topic levels	# of PUBLISH
1	21
2	172
3	162
4	179

And this is the resulting graph.

